Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

1. Chiến lược Kiểm thử & Testcontainers 4
1.1. Chiến lược Kiểm thử trong Microservices 4
1.2. Tháp kiểm thử (Test Pyramid) trong Hệ thống phân tán 4
1.2.1. Giới hạn của E2E Testing truyền thống 5
1.2.2. Dịch chuyển trọng tâm sang Component và Contract Testing 5
1.3. Thay thế Mocking bằng Testcontainers 5
1.3.1. Điểm yếu của In-Memory Databases và Mocking 5
1.3.2. Testcontainers: Môi trường kiểm thử cô lập 5
1.4. Case Study 1: Kiểm thử Idempotency với Kafka 5
1.5. Case Study 2: Kiểm thử chuyển trạng thái khi Saga gặp lỗi 7
1.6. Consumer-Driven Contract Testing 7

1.

CHƯƠNG 1.

Chiến lược Kiểm thử & Testcontainers

Kiểm thử trong kiến trúc phân tán không chỉ là việc xác minh logic nội bộ, mà là việc chứng minh khả năng chống chịu của hệ thống trước độ trễ và lỗi của mạng lưới. Phụ lục này trình bày cách điều chỉnh chiến lược kiểm thử khi chuyển từ monolith sang microservices: từ việc tái cân bằng tháp kiểm thử (Test Pyramid), kiểm thử thành phần với Testcontainers, đến kiểm thử hợp đồng (Contract Testing) giữa các dịch vụ độc lập.

1.1. Chiến lược Kiểm thử trong Microservices

Trong kiến trúc nguyên khối (monolith), quá trình kiểm thử thường tập trung vào logic nghiệp vụ nội bộ và giao tiếp trực tiếp với cơ sở dữ liệu dùng chung. Tuy nhiên, khi chuyển đổi sang kiến trúc microservices, bài toán kiểm thử trở nên phức tạp hơn theo cấp số nhân. Các dịch vụ không còn chia sẻ cùng một không gian bộ nhớ; thay vào đó, chúng giao tiếp thông qua hệ thống mạng với độ trễ và tỷ lệ lỗi không thể dự đoán trước. Việc kiểm thử hệ thống phân tán đòi hỏi một tư duy mới, nơi sự cô lập (isolation) và tính nhất quán (consistency) giữa các hợp đồng API (API Contracts) phải được đặt lên hàng đầu.

1.2. Tháp kiểm thử (Test Pyramid) trong Hệ thống phân tán

Khái niệm tháp kiểm thử (Test Pyramid) được Mike Cohn giới thiệu nhằm tối ưu hóa tỷ lệ giữa chi phí và tốc độ kiểm thử. Tuy nhiên, áp dụng mô hình này vào microservices đòi hỏi sự điều chỉnh chiến lược đáng kể.

1.2.1. Giới hạn của E2E Testing truyền thống

Trong mô hình nguyên khối, kiểm thử từ đầu đến cuối (End-to-End — E2E Testing) hoạt động như cơ chế kiểm soát chất lượng cuối cùng. Nhưng với microservices, việc khởi động toàn bộ cụm dịch vụ (ví dụ: 10 dịch vụ, 3 cơ sở dữ liệu, 1 cụm Kafka) để thực thi bộ kiểm thử E2E là một quy trình tốn kém về tài nguyên và thời gian. Hơn thế nữa, các bài kiểm thử E2E trong môi trường phân tán thường xuyên gặp hiện tượng kết quả không đồng nhất (flaky test) do độ trễ mạng hoặc thứ tự khởi động dịch vụ. Điều này làm giảm đáng kể sự tin cậy của quy trình tích hợp liên tục (CI).

1.2.2. Dịch chuyển trọng tâm sang Component và Contract Testing

Thay vì phụ thuộc vào kiểm thử E2E, chiến lược kiểm thử hiện đại cho microservices tập trung mở rộng hai tầng giữa của kim tự tháp: **Component Testing** (kiểm thử thành phần) và **Contract Testing** (kiểm thử hợp đồng). Component Testing cho phép khởi chạy và kiểm thử một dịch vụ duy nhất trong môi trường cô lập (isolated) bằng cách cung cấp các thành phần hạ tầng (như cơ sở dữ liệu riêng biệt) một cách tạm thời. Song song đó, Contract Testing đảm bảo rằng thông điệp giao tiếp giữa các dịch vụ (ví dụ: giữa API Gateway và Core Service) luôn tuân thủ đúng định dạng đã thỏa thuận mà không cần thiết lập kết nối mạng thực tế giữa chúng trong lúc kiểm thử.

1.3. Thay thế Mocking bằng Testcontainers

Quá trình Component Testing đối mặt với một thách thức lớn: làm thế nào để cung cấp cơ sở dữ liệu và hệ thống thông điệp (message broker) cho dịch vụ trong quá trình chạy kiểm thử tự động?

1.3.1. Điểm yếu của In-Memory Databases và Mocking

Theo truyền thống, các nhà phát triển thường sử dụng cơ sở dữ liệu trong bộ nhớ (in-memory database) như H2 để thay thế cho PostgreSQL, hoặc sử dụng các thư viện giả lập (mock) để thay thế Kafka. Tuy nhiên, chiến lược này tiềm ẩn nhiều rủi ro. Các truy vấn SQL đặc thù của PostgreSQL (như `JSONB` hoặc tính năng khóa bản ghi `FOR UPDATE`) không thể hoạt động chính xác trên H2. Tương tự, việc giả lập hoạt động của Kafka không thể tái hiện các kịch bản thực tế như tái cân bằng nhóm tiêu thụ (consumer group rebalancing) hay xử lý thông điệp trùng lặp (duplicate messages). Sự sai lệch này dẫn đến tình trạng "thành công trong môi trường kiểm thử nhưng thất bại khi triển khai thực tế".

1.3.2. Testcontainers: Môi trường kiểm thử cô lập

Testcontainers là một thư viện mã nguồn mở giúp lập trình viên quản lý các container Docker trực tiếp từ mã kiểm thử (ví dụ: JUnit trong Java). Thay vì sử dụng cơ sở dữ liệu giả lập, Testcontainers giao tiếp với Docker API để khởi động một container PostgreSQL thực tế và một container Kafka thực tế ngay trước khi bộ kiểm thử bắt đầu, và tự động giải phóng tài nguyên (teardown) sau khi kết thúc.

Việc áp dụng Testcontainers mang lại lợi ích kép: đảm bảo độ tin cậy cao do sử dụng cùng một công nghệ hạ tầng với môi trường thực tế, đồng thời giữ được tính tự động hóa và cô lập cần thiết cho quy trình CI/CD. Một bài kiểm thử thành công với Testcontainers mang lại độ tin cậy tương đương với kiểm thử tích hợp thực tế.

1.4. Case Study 1: Kiểm thử Idempotency với Kafka

Idempotency là một yêu cầu kỹ thuật bắt buộc trong giao tiếp bất đồng bộ, nhằm đảm bảo hệ thống không xử lý sai lệch trạng thái khi một thông điệp bị gửi trùng lặp. Việc kiểm thử kịch bản này bằng các công cụ giả lập (mock) thông thường gần như bất khả thi do không phản ánh đúng cơ chế xác nhận thông điệp (offset acknowledgment) của Kafka.

Dưới đây là một minh họa sử dụng Testcontainers để kiểm thử idempotency trong dịch vụ của KBM. Kịch bản yêu cầu hệ thống gửi cùng một sự kiện `SubmissionCreatedEvent` (mang cùng một `eventId`) vào Kafka hai lần liên tiếp, mô phỏng tình huống kết nối mạng không ổn định khiến nhà cung cấp (Producer) phải gửi lại dữ liệu.

```
@SpringBootTest
@Testcontainers
class SubmissionConsumerIdempotencyTest {

    @Container
    static PostgreSQLContainer<?> postgres =
            new PostgreSQLContainer<>("postgres:15-alpine");

    @Container
    static KafkaContainer kafka =
            new KafkaContainer(DockerImageName.parse("confluentinc/cp-kafka:7.5.0"));

    @Autowired
    private KafkaTemplate<String, Object> kafkaTemplate;

    @Autowired
    private SubmissionRepository submissionRepository;

    @DynamicPropertySource
    static void overrideProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.kafka.bootstrap-servers", kafka::getBootstrapServers);
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Test
    void shouldProcessEventOnlyOnce_WhenDuplicateMessagesArrive() throws Exception {
        // Configure message with Idempotency Key (eventId)
        String eventId = UUID.randomUUID().toString();
        SubmissionCreatedEvent event =
                new SubmissionCreatedEvent(eventId, "student123", "question456");

        // Send message the first time
        kafkaTemplate.send("submission-topic", event).get();

        // Send the EXACT SAME MESSAGE a second time (simulate duplicate)
        kafkaTemplate.send("submission-topic", event).get();

        // Wait for Consumer to process (using Awaitility)
        Awaitility.await()
            .atMost(Duration.ofSeconds(5))
            .during(Duration.ofSeconds(2)) // Ensure count remains 1 for at least 2 seconds
            .untilAsserted(() -> {
                // Verify database: data is inserted exactly once
                long count = submissionRepository.countByEventId(eventId);
                assertThat(count).isEqualTo(1);
            });
    }
}
```

Listing 1.1: *Cấu hình và thực thi kiểm thử Idempotency với Kafka Testcontainer*

Kết quả của bài kiểm thử này chứng minh tính đúng đắn trong thiết kế cơ sở dữ liệu và logic xử lý của bên tiêu thụ (Consumer): thông điệp thứ nhất được xử lý thành công, trong khi thông điệp thứ hai bị bỏ qua

nhờ cơ chế ràng buộc duy nhất (unique constraint) dựa trên `eventId` ở cấp độ cơ sở dữ liệu, ngăn chặn tình trạng chấm điểm hai lần cho cùng một bài nộp.

1.5. Case Study 2: Kiểm thử chuyển trạng thái khi Saga gặp lỗi

Saga pattern xử lý các giao dịch phân tán bằng chuỗi các giao dịch cục bộ và các hành động bù trừ (compensation). Việc kiểm thử luồng bù trừ đòi hỏi sự phối hợp khép kín giữa cơ sở dữ liệu trạng thái và hệ thống luân chuyển sự kiện. Kịch bản dưới đây kiểm thử *bước chuyển trạng thái thất bại* — phần dễ kiểm chứng nhất của luồng này: nếu Judge Service gặp sự cố (ví dụ: mã nguồn không hợp lệ), nó phải gửi một sự kiện thất bại về Core Service để Core Service chuyển bài nộp sang trạng thái lỗi. Một bài test compensation đầy đủ hơn còn phải tạo ra side effect trước đó (ví dụ trừ lượt nộp/quota) rồi xác nhận hành động hoàn trả thực sự diễn ra — kể cả khi sự kiện thất bại bị gửi lặp (duplicate) hoặc retry.

```
@Test
void shouldTransitionToError_WhenJudgeServiceEmitsFailureEvent() throws Exception {
  // 1. Core Service receives request and initializes status to JUDGING
  Submission submission = new Submission("sub_1", "student_1", SubmissionStatus.JUDGING);
  submissionRepository.save(submission);

  // 2. Simulate Judge Service emitting a JudgeFailedEvent via Kafka
  JudgeFailedEvent failureEvent = new JudgeFailedEvent("sub_1", "Compilation Error");
  kafkaTemplate.send("judge-result-topic", failureEvent).get();

  // 3. Verify the failure-state transition at Core Service
  Awaitility.await().atMost(Duration.ofSeconds(5)).untilAsserted(() -> {
    Submission updatedSubmission = submissionRepository.findById("sub_1").orElseThrow();
    // Status must move to ERROR instead of stuck in JUDGING
    assertThat(updatedSubmission.getStatus()).isEqualTo(SubmissionStatus.ERROR);
    assertThat(updatedSubmission.getErrorMessage()).isEqualTo("Compilation Error");
  });
}
```

Listing 1.2: *Kiểm thử chuyển trạng thái khi nhận sự kiện thất bại trong Saga*

Bài kiểm thử này không phụ thuộc vào mã nguồn nội bộ của Judge Service. Nó mô phỏng kết quả đầu ra của Judge Service bằng cách trực tiếp phát thông điệp thất bại vào hệ thống Kafka (được Testcontainers cấp phát). Điều này cho phép đội ngũ phát triển Core Service tự tin rằng bài nộp không bị kẹt ở trạng thái trung gian khi Judge báo lỗi, mà không cần thiết lập hệ thống kiểm thử E2E phức tạp.

1.6. Consumer-Driven Contract Testing

Nếu Testcontainers giải quyết bài toán kiểm thử logic nghiệp vụ với hạ tầng thực tế, thì **Contract Testing** giải quyết bài toán giao tiếp giữa các dịch vụ độc lập. Trong microservices, khi Dịch vụ A - bên gọi API (Consumer) gọi Dịch vụ B - bên cung cấp API (Provider), bất kỳ sự thay đổi nhỏ nào trong cấu trúc JSON của Dịch vụ B cũng có thể gây ra lỗi nghiêm trọng tại Dịch vụ A.

Thay vì thực hiện các lời gọi mạng thực tế, kiểm thử hợp đồng hướng tiêu dùng (Consumer-Driven Contract Testing) (sử dụng các công cụ như Pact hoặc Spring Cloud Contract) hoạt động theo nguyên tắc: Consumer định nghĩa trước "kỳ vọng" (contract) về cấu trúc dữ liệu trả về. Hợp đồng này sau đó được tự động chia sẻ và chạy trên hệ thống tích hợp liên tục (CI) của Provider. Nếu Provider thay đổi mã nguồn làm vi phạm hợp đồng (ví dụ: đổi tên trường `studentId` thành `userId`), bộ kiểm thử của Provider sẽ thất bại ngay lập tức, ngăn chặn việc triển khai phiên bản lỗi lên môi trường thực tế. Chiến lược này giảm tải đáng kể cho hệ thống kiểm thử E2E, đồng thời nâng cao tính tự chủ cho các nhóm phát triển độc lập.